

W2 PRACTICE

Native HTTP and Manual Routing



At the end of this practice, you can

- ✓ **Create** and run a native Node.js HTTP server ✓
Manually implement route handling using conditionals.
- ✓ Serve **static files** using fs. ✓ Parse form data from POST requests.
- ✓ Debug and enhance server code using console outputs.



Get ready before this practice!

- ✓ **Read** the following documents to understand Nodejs built-in HTTP module:
<https://nodejs.org/api/http.html>
- ✓ **Read** the following documents to understand Anatomy of an HTTP Transaction:
<https://nodejs.org/en/learn/modules/anatomy-of-an-http-transaction>



How to submit this practice?

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



EXERCISE 1 ANALYZE

Goal

- ✓ Identify and fix the bug.
- ✓ Understand the request-response cycle in Node.js using the `http` module.
- ✓ Explain the role of `res.write()` and `res.end()` in sending data back to the client.



For this exercise, you are provided with a minimal `server.js` file. Read and run the code and observe how it behaves.

```
// server.js
const http = require('http');
const server = http.createServer((req, res) => {
  res.write('Hello, World!'); return
  res.endd();
}); server.listen(3000, () => { console.log('Server
running on http://localhost:3000'); });
```

Q1 – What error message do you see in the terminal when you access `http://localhost:3000`? What line of code causes it?

Error message, we see in the terminal when we access `http://localhost:3000` is `TypeError: res.endd is not a function`

This error occurs because of a typo in the code: `res.endd()` is incorrect. The correct method is

`res.end()`.

Error line: `return res.endd();`

Q2 – What is the purpose of `res.write()` and how is it different from `res.end()`?

- `res.write()` sends a chunk of data to the client **without ending** the response.
- `res.end()` signals that **all data has been sent** and the response is complete.

Q3 – What do you think will happen if `res.end()` is not called at all?

If `res.end()` is not called, the server will not close the response. The client (e.g., a web browser) will continue waiting for the response to finish, possibly resulting in a timeout or hanging.

Q4 – Why do we use `http.createServer()` instead of just calling a function directly?

`http.createServer()` is used to create an actual HTTP server that listens for client requests. Just calling a function directly doesn't start a server or listen to HTTP requests. `http.createServer()` binds the function to handle each request and opens a listening port.

Q5 – How can the server be made more resilient to such errors during development?

To make the server more resilient:

1. Use a **linter** (like ESLint) to detect typos and syntax errors.
2. Use **TypeScript** for type safety and better editor support.
3. Use **try-catch** blocks or global error handling with `process.on('uncaughtException', handler)`.
4. Use tools like **Nodemon** to auto-restart the server and show real-time errors.
5. Write **unit tests** to catch bugs early.

EXERCISE 2 MANIPULATE

Goal

- ✓ Practice using `req.url` and `req.method`.
- ✓ Understand how manual routing mimics what frameworks (like Express) automate. ✓
Serve both plain text and raw HTML manually.

🔗 For this exercise you will start with a **START CODE (EX-2)**

TASK 1 - Update the code above to add custom responses for these routes:

ROUTE	HTTP METHOD	RESPONSE
/about	GET	About us: at CADT, we love node.js!
/contact-us	GET	You can reach us vai email...
/products	GET	Buy one get one...
/projects	GET	Here are our awesome projects

Use [VS Code's Thunder Client](#) (or other tools (POSTMAN, INSOMIA) of your choice or curl on your terminal to make request.

Example output

```
curl http://localhost:3000/about ----->
About us: at CADT, we love node.js!
```

```
curl http://localhost:3000/contact-us -----
-> You can reach us vai email...
```

TASK 2 – As we can see the complexitiy grow as we add more routes. Use `switch` statement to arrange the code into more organized structure.

🔗 Reflective Questions

1. What happens when you visit a URL that doesn't match any of the three defined?

The server will return a default response or a 404 Not Found message if coded; otherwise, it may hang or return nothing, depending on implementation.

2. Why do we check both the `req.url` and `req.method`?

Because the same URL can be used for different actions depending on the HTTP method (e.g., GET, POST). Checking both ensures accurate routing and behavior.

3. What MIME type (`Content-Type`) do you set when returning HTML instead of plain text?

- `text/html`

4. How might this routing logic become harder to manage as routes grow?

The number of if-else or switch statements increases, making the code harder to read, maintain, and debug. It also increases the chances of errors or duplication.

5. What benefits might a framework offer to simplify this logic?

Frameworks like Express provide:

- Cleaner and declarative route definitions
- Middleware support
- Built-in error handling
- Easier parsing of request data
- Improved scalability and maintainability

EXERCISE 3 CREATE

Goal

✓ Practice handling `POST` requests. ✓ Parse URL-encoded form data manually. ✓ Write and append to local files using Node.js' `fs` module. ✓ Handle async operations and errors gracefully.

🔧 For this exercise you will start with a **START CODE EX-3**

TASK 1 - Extend your Node.js HTTP server to handle a **POST request** submitted from the contact form. When a user submits their name, the server should:

1. **Capture the form data** (from the request body).
2. **Log it to the console.**
3. **Write it to a local file** named `submissions.txt`.

Testing, go to `/contact` on browser and test

Requirements • Handle `POST /contact`

requests.

- Parse raw `application/x-www-form-urlencoded` data from the request body.
- Write the name to a new line in `submissions.txt`.
- Send a success response to the client (HTML or plain text).

? Discussion Questions

1. Why do we listen for `data` and `end` events when handling `POST`?

Because `POST` request data is streamed in chunks. We use `data` to collect those chunks and `end` to know when the full body has been received.

2. What would happen if we didn't buffer the body correctly?

The form data would be incomplete or corrupted, leading to incorrect or failed parsing.

3. What is the format of form submissions when using the default browser form `POST`?

It's `application/x-www-form-urlencoded`, which encodes data as key-value pairs (e.g., `name=John+Doe`).

4. Why do we use `fs.appendFile` instead of `fs.writeFile`?

`fs.appendFile` adds data to the end of the file without overwriting existing content, preserving previous submissions.

5. How could this be improved or made more secure?

Improvements include:

- Validating and sanitizing input to prevent injection attacks
- Using `HTTPS` to encrypt form data
- Storing data in a database instead of plain text
- Adding error handling and input size limits



Bonus Challenge (Optional)

- Validate that the `name` field is not empty before saving.
- Send back a small confirmation HTML page instead of plain text.
- Try saving submissions in JSON format instead of plain text.